Alisa Sample

Relevant database tables

Projects/Generated Work

- **Projects table** - holds the users input and generated image
- **Jobs** - tracks the work for a project
- **Files** - stores file metadata and storage path referencing job id

AI Chat and Usage

- **Chat_sessions** - stores chat history
- **Chat_messages** - stores past chat messages and uses chat_sessions id
- **Message_feedback** - stores message feedback
- **User_subscriptions** - tracks balances

Exports and Activity

- **Export_jobs** - export lifecycle table, links to project, artifact file, and compliance record
- **Activity_events** - stores export lifecycle events
- **Notifications** - stores per-user notifications

Compliance

- **Jurisdiction, compliance_presets, compliance_rules, project_compliance_records, and compliance_exceptions** - all support jurisdiction-based validation and review

Teams

- **Workspaces** - represents a team/workspace
- **Workspace_members** - associates users and roles
- **Workspace_invites** - stores invitations, tokens, and invite status

Relevant APIs

- **Gemini** - transcribes audio file, generates an image using enhanced prompt from CADLLM
- **CADLLM** - uses transcribed text to enhance prompt
- **Api2Convert** - converts image generated from gemini to DXF file
- **FastAPI** - helps build validated, structured, and maintainable API applications

Existing Activity functionality

- **Persistent feed** - uses activity_events. Activity_service.py records and reads events. Worker_service.py emits events
- **Dashboard feed** - derives activity from recent jobs and their statuses. Does not read activity_events

Where file uploads happen

Audio Transcription

- **ToolbarComponenets.tsx** - accepts audio and sends it via api.ts
- **Recorded Audio** - uses useAudioWaves.ts
- **Chat recording/ transcription** - uses ChatInterface.tsx and chat.ts - the backend transcriber writes a temporary file, uploads it to gemini for transcription, and deletes the file afterward without saving it as a project file

Where exports happen

- **Export_jobs** - created by AP - records in queued status and provides reads, status updates, and artifact lookup
- **Worker_service.py** - background poller and status/event transitions. Its generation hook calls export_pipeline_service.py where export generation is currently incomplete. It does not generate or attach an artifact
- **Exports UI** - exports/page.tsx. Gathers exports project-by-project, displays status, and requests URLS for downloads.

Where AI actions happen

- **Chat_routes.py** - handles main chat generation flow: check request balance, creates/ loads a session, saves the user message, enhances the prompt, generates image, can run US IRC compliance extraction, saves the ai response, links the session to a project, and deducts usage
- **Chat_model.py** - chat data shapes
- **transcriber.py** - handles audio to text but runs a 501 error
- **Dxf_generator** - runs the audio/text-to-CAD pipeline.
- The service implementations are in gemini_transcription_service.py, gemini_service.py, and the imported CAD/conversion services called from dxf_generator.py.

Where team actions happen

- **members/page.tsx** - loads the current workspace and summary
- **MemberManagement.tsx** - loads members/invites and handles role changes, removals, invite creation, and invite revocation

    - InviteForm.tsx - validates invite input
    - MembersList.tsx and PendingInvites.tsx - render respective lists
- **Workspace_routes.py, workspace_queries.py, invite_routes.py, and invite_queries.py** - backend authorization and persistence